WEB3 — HETIC

ENSO Monitor

Conception et développement d’une plateforme d’analyse d’El Niño et de La Niña

SUJET DE RATTRAPAGE — 5 JOURS

RNCP 36146 · Concepteur développeur de solutions digitales · Niveau 6

Priorité de l’évaluation — code review individuelle

La code review occupe une place prépondérante dans l’appréciation pédagogique. Chaque étudiant devra défendre à l’oral son code, l’architecture, la conception, les tests, l’organisation et les décisions de gestion de projet. L’intelligence artificielle est autorisée, mais la compréhension et la justification personnelles sont indispensables.

HETIC — WEB3 | Projet ENSO Monitor | 2026

PARTIE 01 / MISSION

Contexte, objectif et périmètre

Ce rattrapage est strictement individuel. Vous devez concevoir seul une application web permettant d'explorer des données scientifiques réelles relatives aux épisodes El Niño et La Niña. Le produit doit transformer les observations climatiques en visualisations compréhensibles, sourcées et vérifiables.

MVP obligatoire — quatre fonctionnalités

- Importer une série NOAA authentique (ONI ou RONI, au choix) dans PostgreSQL avec Python ; identifier la source et contrôler les doublons.
- Exposer les observations par une API FastAPI documentée, avec filtrage temporel et validation des entrées.
- Visualiser les valeurs dans React / TypeScript au moyen d'une courbe interactive et d'un filtre de période.
- Comparer deux épisodes historiques à partir des mesures disponibles et présenter une interprétation prudente.

Les fonctionnalités complémentaires (administration, automatisation, cartographie, statistiques avancées) sont facultatives. Un MVP réduit, cohérent, testé et compris est préférable à une application plus riche mais mal maîtrisée.

Données scientifiques — ONI ou RONI au choix

Une seule série est obligatoire : choisissez l'indice NOAA ONI ou RONI. Il n'est pas demandé de traiter les deux. Documentez la source, la version et la date d'extraction. La comparaison de deux épisodes historiques utilise l'indice retenu.

SOURCE	USAGE	PRÉCAUTION
NOAA CPC — RONI	Suivi des phases ENSO et comparaison temporelle.	Préciser période, version, date d'extraction.
NOAA CPC — ONI	Étude historique des anomalies de la région Niño 3.4.	Ne pas confondre ONI et RONI.
Copernicus ERA5 (bonus)	Approfondissement géographique.	Justifier l'origine et les agrégations.

Le terme « Super El Niño » n'est pas une classe scientifique universelle. Toute classification utilisée doit indiquer les seuils, la durée et l'indice retenu. Aucune prévision climatique n'est demandée.

Deux livrables distincts

Livrable 1 : application et code source. Livrable 2 : dossier de conception structuré et détaillé de 15 pages maximum, couverture comprise. La procédure de déploiement est incluse dans le dossier, mais le déploiement réel est facultatif.

PARTIE 02 / APPLICATION

Exigences fonctionnelles et techniques

Interface — exigences du MVP

- Une courbe temporelle issue d'un indice NOAA ; période sélectionnable, unités et source visibles.
- Comparaison de deux épisodes historiques, avec une explication écrite des résultats et limites.
- Une interface responsive avec légende lisible, navigation utilisable au clavier et prise en charge des états d'erreur.

L'identification automatique des phases, les tableaux d'administration, les exports et les statistiques avancées sont des extensions, non des critères d'échec du MVP.

Socle d'architecture

COUCHE	STACK	ATTENDUS
Frontend	React, TypeScript, Vite	Composants réutilisables, typage strict, état maîtrisé.
API	Python, FastAPI	Validation Pydantic, OpenAPI, routes /api/v1, codes HTTP pertinents.
Données	PostgreSQL, SQLAlchemy, Alembic	Contraintes SQL, schéma cohérent, migrations.
ETL	Python, Pandas, NumPy	Imports fiables, gestion des doublons, traçabilité NOAA.
Tests	Pytest, Vitest / Playwright	3 tests : transformation, API et parcours frontend.

Tests minimaux à exécuter et documenter

- Test de données : vérifier une transformation Python sur des observations NOAA (ex. doublons ou date invalide).
- Test API : vérifier une route FastAPI filtrée par période, avec réponse attendue et traitement d'une entrée invalide.
- Test frontend : vérifier qu'un changement de période actualise la courbe ou les valeurs affichées.

Qualité et UX

- Architecture lisible ; conventions Git, README, linting et séparation des responsabilités.
- Responsive mobile/desktop, contrastes et navigation au clavier, graphiques accessibles.
- Requêtes paramétrées, secrets hors Git, contrôle d'accès si une administration existe.
- Éviter les fonctionnalités accessoires non maîtrisées : la qualité du MVP prime sur son volume.

PARTIE 03 / DOSSIER DE CONCEPTION

Document indépendant — 15 pages maximum

Un document distinct du code et de l’application devra démontrer que vous savez concevoir, argumenter et organiser une solution digitale. Il ne s’agit pas d’un simple compte rendu descriptif : chaque section doit expliquer les décisions et leurs conséquences.

Structure imposée et volume indicatif

SECTION	CONTENU OBLIGATOIRE	PAGES
1. Besoin et contexte	Objectif, public cible, contraintes, périmètre du MVP, limites du sujet.	1–2
2. Fonctionnel et UX	User stories, priorisation, parcours utilisateurs, wireframes et critères d’acceptation.	2
3. UML et modélisation	Diagramme de cas d’utilisation ; diagramme de classes ou modèle conceptuel ; diagramme de séquence d’un parcours clé.	3
4. Architecture technique	Schéma frontend/API/base/ETL ; composants, flux de données, décisions d’architecture, schéma SQL.	2–3
5. Organisation du projet	Backlog personnel, planning sur cinq jours, priorités, outils, risques, arbitrages et suivi.	1–2
6. Qualité et sécurité	Stratégie de tests, vérifications UX, sécurité, performance et traitement des anomalies.	1–2
7. Déploiement proposé	Architecture cible VPS, Docker Compose, Caddy, réseaux, configuration, sauvegardes et limites.	1–2
8. Bilan et sources	Choix justifiés, apports de l’IA, difficultés, corrections, sources scientifiques.	1

La limite totale est de 15 pages maximum. Les volumes par section sont indicatifs et doivent être adaptés sans dépasser le plafond. Les diagrammes doivent être lisibles et commentés, pas seulement insérés.

PARTIE 04 / DIAGRAMMES ET JUSTIFICATION

Une conception démontrable à l’oral

Diagrammes UML attendus

- Cas d’utilisation : acteurs (visiteur, administrateur si utile), actions principales et frontière du système.
- Classes ou modèle de domaine : entités Observation, Indice, Source, Import et leurs relations ; justification des cardinalités.
- Séquence : chemin d’une consultation de graphique, du navigateur à l’API puis à PostgreSQL, réponse et erreurs.

Schémas d’architecture complémentaires

- Diagramme d’architecture logique : React, FastAPI, PostgreSQL et worker ETL ; responsabilités et échanges.
- Schéma relationnel : tables, clés, index utiles, intégrité, migrations et provenance des observations.
- Diagramme ou procédure de déploiement cible : réseau Docker interne, point d’entrée Caddy, flux HTTPS, volumes et sauvegardes.

Argumentation obligatoire

Pour chaque choix déterminant, précisez le besoin, les solutions envisagées, le choix final, les avantages, les limites et la manière de vérifier le résultat. Exemples : pourquoi PostgreSQL, pourquoi un traitement ETL séparé, pourquoi une API versionnée, pourquoi Docker Compose, comment assurer l’accessibilité du graphique et la traçabilité des données.

Épreuve orale

Le jury peut demander de commenter un diagramme UML, d’expliquer les dépendances, de modifier une route API, de reconstruire un flux de données ou d’exposer les conséquences d’une mauvaise décision technique ou organisationnelle.

PARTIE 05 / DÉPLOIEMENT

Documentation obligatoire, mise en ligne facultative

Aucun investissement financier n’est demandé aux étudiants. Il n’est pas nécessaire de louer un VPS, d’acheter un nom de domaine, de payer une adresse IP ou de souscrire un service cloud. La mise en production effective est facultative et n’intervient pas comme condition obligatoire de validation.

Ce que doit contenir le dossier de conception

- Solution d’hébergement proposée : VPS Linux chez un hébergeur au choix (Hetzner à titre d’exemple), avec dimensionnement estimé et justification, sans achat.
- Conteneurisation : plan Docker Compose pour frontend, API, PostgreSQL et worker ETL ; dépendances, volumes et réseaux internes.
- Reverse proxy : rôle de Caddy, routage, certificats HTTPS, ports exposés et politique de sécurité.
- Configurations : variables d’environnement, secrets, accès SSH, pare-feu, maintenance et procédure de mise à jour.
- Exploitation : sauvegardes, restauration, journalisation, supervision, retour arrière et gestion d’une panne.
- Capacité et limites : dimensionnement CPU/RAM du VPS, limites d’un seul serveur et pistes simples d’optimisation.

Preuves acceptées sans hébergement payant

Schéma d’infrastructure, extraits de **compose.yaml** et de **Caddyfile**, procédure de déploiement pas à pas, démonstration en local avec Docker et captures de tests. Une présentation des commandes de mise en production et de restauration suffit si elle est techniquement justifiée.

Distinction essentielle

Documenter et défendre la solution de déploiement : obligatoire.
Déployer réellement sur un VPS ou Internet : facultatif.
Dépense personnelle : jamais exigée.

PARTIE 06 / SOUTENANCE

Code review et justification individuelle

La code review individuelle est un élément central de la soutenance. L'enseignant analysera non seulement le code, mais aussi la conception, les diagrammes UML, l'organisation du travail, les tests, la sécurité et la proposition de déploiement.

Principe de la soutenance

La quantité de fonctionnalités produites n'est pas le critère prioritaire. En cinq jours, vous devrez démontrer la cohérence de la solution, la qualité de la conception, les tests réellement effectués et la capacité de chaque étudiant à défendre personnellement ses décisions. Une solution sobre et maîtrisée est préférable à un ensemble de fonctionnalités nombreuses, fragiles ou incomprises.

Utilisation de l'intelligence artificielle

L'IA générative est autorisée pour rechercher, planifier, coder, tester, documenter et déboguer. Son usage ne dispense jamais de la compréhension personnelle : tout étudiant doit pouvoir expliquer le résultat, les alternatives et les compromis, puis modifier le travail produit.

- Déclarer les outils IA utilisés et les principales parties assistées dans le dossier ou le README.
- Justifier les décisions techniques et organisationnelles, même lorsqu'elles proviennent d'une suggestion de l'IA.
- Savoir reproduire un bug, expliquer le correctif et exécuter les tests.
- Ne pas présenter comme réel un résultat climatique fictif, ni comme exécuté un test non réalisé.

Points à pouvoir justifier à l'oral

AXE	DÉMONSTRATION ATTENDUE
BC01 — conception	Expliquer besoins, spécifications, UML, modèle de données et architecture.
BC02 / C2.2 — cycle de vie	Justifier backlog, priorités, arbitrages, risques et tests.
BC03 / C3.1 — programmation	Parcourir son code, expliquer chaque partie significative et les pratiques de sécurité.
BC03 / C3.2 — outils	Défendre React, FastAPI, PostgreSQL, ETL et l'architecture de déploiement.
BC03 / C3.3 — correction	Identifier une anomalie, la corriger et vérifier la non-régression.

PARTIE 07 / REMISE ET SOURCES

Livrables et vérification finale

Livrable A — application

- Dépôt Git avec sources du frontend, de l’API et de l’ETL ; scripts de base et données sourcées.
- README : prérequis, lancement local, variables attendues, jeux de données et tests.
- Preuves des trois tests minimaux et historique Git montrant la progression personnelle.

Livrable B — dossier de conception (15 pages maximum)

- Document PDF indépendant, structuré, paginé et doté d’un sommaire.
- Diagrammes UML obligatoires et expliqués, maquettes, architecture et modèle de données.
- Planification sur cinq jours, justification des choix et stratégie de tests/cybersécurité.
- Section de déploiement cible documentée, sans obligation de déployer.
- Explications des apports de l’IA, limites, arbitrages et retour d’expérience.

Conditions de réussite

Une application fonctionnelle sans compréhension de l’architecture, du code ou des décisions d’organisation ne suffit pas. Un dossier de qualité ne remplace pas non plus le développement attendu. La priorité est donnée à une solution cohérente et maîtrisée, défendue individuellement à l’oral, et non au nombre de fonctionnalités produites.

Références de travail

France compétences — RNCP 36146 : https://www.francecompetences.fr/recherche/rncp/36146

NOAA — RONI : https://www.cpc.ncep.noaa.gov/products/analysis_monitoring/enso/roni/

NOAA — ONI : https://www.cpc.ncep.noaa.gov/products/analysis_monitoring/enso/oni/v6/index.php

Caddy : https://caddyserver.com/docs/

Docker Compose : https://docs.docker.com/compose/

Le présent document précise les réalisations et les justifications attendues pour ce sujet de rattrapage individuel.